

UART Transceiver Design on FPGA

Full-Duplex Serial Communication using Verilog FSMs

Author : Shashwat Saha

Abstract

This report presents the design, simulation, and FPGA synthesis of a full-duplex UART (Universal Asynchronous Receiver/Transmitter) transceiver implemented in Verilog. The design incorporates a baud rate generator, a transmitter FSM, and a receiver FSM with 16x oversampling, targeting 115,200 baud operation from a 50 MHz system clock. The design was verified through a loopback testbench achieving 100% data integrity across 256 transmitted bytes (0x00 to 0xFF). Post-implementation results on the Zynq-7000 FPGA confirm 49 LUTs, 56 flip-flops, and a maximum operating frequency of 220 MHz, with total on-chip power of 0.093 W.

1. Introduction

UART is one of the most widely used serial communication protocols in embedded systems and digital hardware design. It enables asynchronous point-to-point communication between devices without a shared clock, relying instead on agreed baud rates and framing conventions. This project implements a complete UART transceiver in synthesisable Verilog, targeting deployment on a Xilinx Zynq-7000 FPGA.

The system architecture consists of four modules: a baud rate generator, a transmitter, a receiver, and a top-level integration module. The receiver employs 16x oversampling for robust start-bit detection and mid-bit sampling, reducing susceptibility to timing skew. The design was simulated using a loopback testbench in Xilinx Vivado XSim and subsequently synthesised and implemented for the target FPGA.

1.1 Objectives

- Implement a full-duplex UART transceiver at 115,200 baud with a 50 MHz system clock
- Design transmitter and receiver as separate finite state machines (FSMs)
- Verify loopback data integrity across all 256 possible byte values (0x00 to 0xFF)
- Synthesise and implement the design on Zynq-7000, reporting LUT, FF, and timing results

2. System Overview

The UART system is structured as four Verilog modules, each with a distinct responsibility. The top-level uart module instantiates the baud rate generator, transmitter, and receiver, and routes internal signals between them.

2.1 System Architecture

Module	Description
uart.v	Top-level integration module; routes data, clock enables, and control signals
baudrate.v	Divides 50 MHz clock to produce Tx (115,200 Hz) and Rx (1,843,200 Hz) enables
transmitter.v	4-state FSM: IDLE → START → DATA → STOP; serialises 8-bit data LSB-first
receiver.v	4-state FSM: IDLE → START → DATA → STOP; deserialises with 16x oversampling

2.2 Baud Rate Generation

The baud rate generator uses two independent accumulators. The transmitter accumulator counts to $TX_ACC_MAX = 50,000,000 / 115,200 = 434$ clock cycles per bit, generating a single-cycle enable pulse ($Txclk_en$) at 115,200 Hz. The receiver accumulator counts to $RX_ACC_MAX = 50,000,000 / (115,200 \times 16) = 27$ clock cycles, producing $Rxclk_en$ at 1,843,200 Hz — 16 pulses per UART bit period — to enable oversampled reception.

Parameter	Value
System Clock	50 MHz
Baud Rate	115,200 bps
TX_ACC_MAX	434 cycles
RX_ACC_MAX	27 cycles
Oversampling Factor	16x
Bit Period	8,680 ns

3. FSM Design

3.1 Transmitter FSM

The transmitter implements a Moore FSM with four states. On assertion of Tx_en, the FSM captures the 8-bit input data and transitions from IDLE to START. Each state transition in START, DATA, and STOP is gated by Txclk_en to maintain correct baud timing. Data bits are transmitted LSB-first. The Tx_busy output is asserted whenever the FSM is not in IDLE, allowing the testbench to detect transmission completion.

State	Behaviour
TX_IDLE (00)	Tx=1 (line idle). Transitions to START when Tx_en=1; latches data_in
TX_START (01)	On Txclk_en: drives Tx=0 (start bit), moves to DATA
TX_DATA (10)	On each Txclk_en: drives data[bit_pos], increments bit_pos. After bit 7, moves to STOP
TX_STOP (11)	On Txclk_en: drives Tx=1 (stop bit), returns to IDLE

3.2 Receiver FSM

The receiver uses 16x oversampling. In the IDLE state it monitors the Rx line for a falling edge (start bit). Upon detection it enters START and counts 8 Rxclk_en pulses to reach the midpoint of the start bit, where it confirms the line is still low before proceeding. In DATA state it counts 16 Rxclk_en pulses between samples, sampling each bit at its midpoint. After 8 bits, it enters STOP and waits for the stop bit to confirm a valid frame before asserting ready and latching data.

State	Behaviour
RX_IDLE (00)	Monitors Rx. On falling edge, enters START with sample=0
RX_START (01)	Counts 8 Rxclk_en pulses to reach start-bit midpoint. If Rx still low: enter DATA. Else: false start, return to IDLE

RX_DATA (10)	Counts 16 Rxclk_en pulses per bit. Samples Rx at sample==15. After 8 bits enters STOP
RX_STOP (11)	Counts 16 Rxclk_en pulses. If Rx=1 at midpoint: latch scratch to data, assert ready. Return to IDLE

4. Verilog Source Code

4.1 baudrate.v

```

module baudrate(
    input  wire clk_50m,
    output wire Rxclk_en,
    output wire Txclk_en
);
    parameter TX_ACC_MAX    = 50000000 / 115200;    // 434
    parameter RX_ACC_MAX    = 50000000 / (115200*16); // 27
    parameter TX_ACC_WIDTH = $clog2(TX_ACC_MAX);
    parameter RX_ACC_WIDTH = $clog2(RX_ACC_MAX);

    reg [RX_ACC_WIDTH-1:0] rx_acc = 0;
    reg [TX_ACC_WIDTH-1:0] tx_acc = 0;

    assign Rxclk_en = (rx_acc == 0);
    assign Txclk_en = (tx_acc == 0);

    always @(posedge clk_50m) begin
        if (rx_acc == RX_ACC_MAX-1) rx_acc <= 0;
        else rx_acc <= rx_acc + 1;
    end
    always @(posedge clk_50m) begin
        if (tx_acc == TX_ACC_MAX-1) tx_acc <= 0;
        else tx_acc <= tx_acc + 1;
    end
endmodule

```

4.2 transmitter.v

```

module transmitter(
    input  wire [7:0] data_in,
    input  wire Tx_en,
    input  wire clk_50m,
    input  wire clken,
    output reg Tx,
    output wire Tx_busy
);
    initial Tx = 1'b1;

    parameter TX_STATE_IDLE   = 2'b00;
    parameter TX_STATE_START  = 2'b01;
    parameter TX_STATE_DATA   = 2'b10;
    parameter TX_STATE_STOP   = 2'b11;

    reg [7:0] data    = 8'h00;
    reg [2:0] bit_pos  = 3'h0;
    reg [1:0] state    = TX_STATE_IDLE;

```

```

always @(posedge clk_50m) begin
    case (state)
        TX_STATE_IDLE: begin
            Tx <= 1'b1;
            if (Tx_en) begin
                state <= TX_STATE_START;
                data <= data_in;
                bit_pos <= 3'h0;
            end
        end
        TX_STATE_START: begin
            if (clken) begin Tx <= 1'b0; state <= TX_STATE_DATA; end
        end
        TX_STATE_DATA: begin
            if (clken) begin
                Tx <= data[bit_pos];
                bit_pos <= bit_pos + 1'b1;
                if (bit_pos == 3'h7) state <= TX_STATE_STOP;
            end
        end
        TX_STATE_STOP: begin
            if (clken) begin Tx <= 1'b1; state <= TX_STATE_IDLE; end
        end
        default: begin Tx <= 1'b1; state <= TX_STATE_IDLE; end
    endcase
end
assign Tx_busy = (state != TX_STATE_IDLE);
endmodule

```

4.3 receiver.v

```

module receiver(
    input wire      Rx,
    input wire      Rx_en,
    output reg      ready,
    input wire      ready_clr,
    input wire      clk_50m,
    input wire      clken,
    output reg [7:0] data
);
    initial begin ready = 1'b0; data = 8'h00; end

    parameter RX_STATE_IDLE   = 2'b00;
    parameter RX_STATE_START  = 2'b01;
    parameter RX_STATE_DATA   = 2'b10;
    parameter RX_STATE_STOP   = 2'b11;

    reg [1:0] state = RX_STATE_IDLE;
    reg [3:0] sample = 4'd0;
    reg [3:0] bit_pos = 4'd0;
    reg [7:0] scratch = 8'd0;

    always @(posedge clk_50m) begin
        if (ready_clr) ready <= 1'b0;
        if (clken && Rx_en) begin
            case (state)
                RX_STATE_IDLE: begin
                    if (!Rx) begin state <= RX_STATE_START; sample <= 4'd0;

```

```

        if (!Rx) begin
            state <= RX_STATE_DATA; sample <= 4'd0;
            bit_pos <= 4'd0; scratch <= 8'd0;
        end else begin
            state <= RX_STATE_IDLE; sample <= 4'd0;
        end
        end else sample <= sample + 1'b1;
    end
    RX_STATE_DATA: begin
        if (sample == 4'd15) begin
            scratch[bit_pos[2:0]] <= Rx;
            sample <= 4'd0;
            bit_pos <= bit_pos + 1'b1;
            if (bit_pos == 4'd7) state <= RX_STATE_STOP;
        end else sample <= sample + 1'b1;
    end
    RX_STATE_STOP: begin
        if (sample == 4'd15) begin
            if (Rx) begin data <= scratch; ready <= 1'b1; end
            state <= RX_STATE_IDLE; sample <= 4'd0;
        end else sample <= sample + 1'b1;
    end
    default: state <= RX_STATE_IDLE;
endcase
end
end
endmodule

```

4.4 uart.v (Top-Level)

```

module uart(
    input wire [7:0] data_in,
    input wire Tx_en, clear, clk_50m,
    output wire Tx, Tx_busy,
    input wire Rx, Rx_en,
    output wire ready,
    input wire ready_clr,
    output wire [7:0] data_out,
    output [7:0] LEDR,
    output wire Tx2
);
    assign LEDR = data_in;
    assign Tx2 = Tx;
    wire Txclk_en, Rxclk_en;
    baudrate
    uart_baud(.clk_50m(clk_50m), .Rxclk_en(Rxclk_en), .Txclk_en(Txclk_en));
    transmitter uart_Tx(.data_in(data_in), .Tx_en(Tx_en), .clk_50m(clk_50m),
        .clken(Txclk_en), .Tx(Tx), .Tx_busy(Tx_busy));

    receiver
    uart_Rx(.Rx(Rx), .Rx_en(Rx_en), .ready(ready), .ready_clr(ready_clr),
        .clk_50m(clk_50m), .clken(Rxclk_en), .data(data_out));
endmodule

```

5. Testbench

5.1 Testbench Design

The testbench instantiates the `uart` module in loopback configuration — the Tx output is connected directly to the Rx input. A 50 MHz clock is generated with a 10 ns half-period. The `send_byte` task asserts `Tx_en`, waits for `Tx_busy` to go high (transmission started), de-asserts `Tx_en`, waits for `Tx_busy` to go low (transmission complete), then waits for the ready flag from the receiver. It then displays a PASS/FAIL result, asserts `ready_clr` for one cycle, and waits a short inter-byte gap before the next transmission.

The full test loops through all 256 byte values (0x00 to 0xFF) and reports a final summary with pass count, fail count, and data integrity percentage.

5.2 `uart_tb.v`

```
module uart_tb();
    reg [7:0] data;
    reg clk=0, enable=0, Rx_en=0, ready_clr=0;
    wire Tx_busy, ready;
    wire [7:0] Rx_data;
    wire loopback;

    uart test_uart(
        .data_in(data), .Tx_en(enable), .clk_50m(clk),
        .Tx(loopback), .Tx_busy(Tx_busy), .Rx(loopback),
        .ready(ready), .ready_clr(ready_clr),
        .Rx_en(Rx_en), .data_out(Rx_data)
    );
    always #10 clk = ~clk; // 50 MHz

    task send_byte;
        input [7:0] byte;
        begin
            data = byte; enable = 1;
            wait (Tx_busy == 1);
            enable = 0;
            wait (Tx_busy == 0);
            wait (ready == 1);
            $display("Sent: %h Received: %h %s",
                byte, Rx_data,
                (Rx_data == byte) ? "PASS" : "FAIL");
            ready_clr = 1; #40 ready_clr = 0; #500;
        end
    endtask

    integer i, pass_count, fail_count;
    initial begin
        $dumpfile("uart.vcd");
        $dumpvars(0, uart_tb);
        pass_count = 0; fail_count = 0;
        Rx_en = 1; #200;
        for (i = 0; i < 256; i = i + 1) begin
            data = i[7:0]; enable = 1;
            wait (Tx_busy == 1); enable = 0;
            wait (Tx_busy == 0);
            wait (ready == 1);
            if (Rx_data == i[7:0]) pass_count = pass_count + 1;
            else fail_count = fail_count + 1;
            $display("Sent: %h Received: %h %s",
```

```

        i[7:0], Rx_data,
        (Rx_data==i[7:0]) ? "PASS" : "FAIL");
    ready_clr = 1; #40 ready_clr = 0; #500;
end
$display("-----");
$display("TOTAL: %0d PASS, %0d FAIL out of 256 bytes",
        pass_count, fail_count);
$display("Data integrity: %0d%%", (pass_count*100)/256);
$finish;
end
endmodule

```

6. Simulation Results

6.1 Waveform — Loopback Operation

The waveform below shows the UART loopback transmission at the 200 μ s time scale. Key signals — clk, enable, Rx_en, Tx_busy, ready, loopback (Tx→Rx), and Rx_data — confirm correct sequential operation of the transmitter and receiver FSMs.

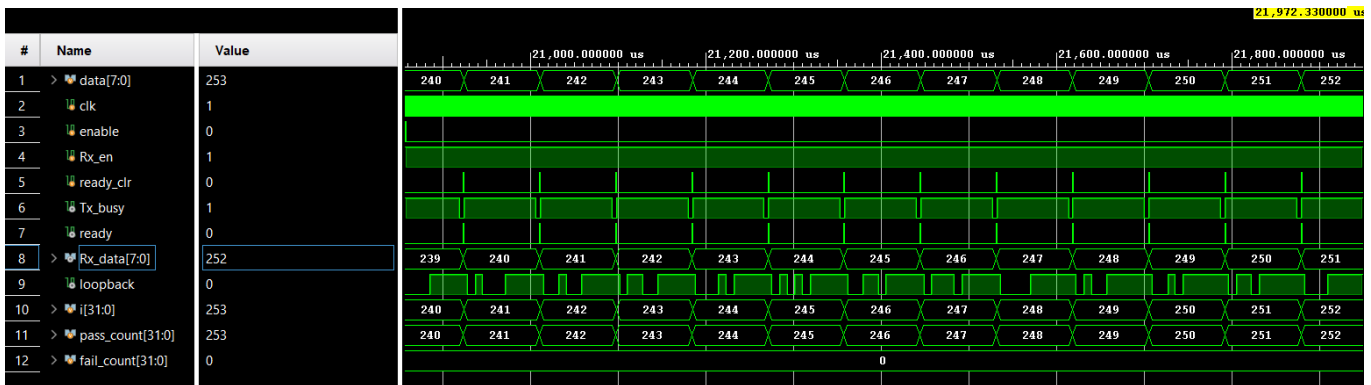


Figure 1: UART Loopback Waveform

6.2 TCL Console — 256-Byte Integrity Test

The testbench was run with all 256 byte values (0x00 to 0xFF). The Vivado XSim TCL console output confirms 100% data integrity with zero failures.

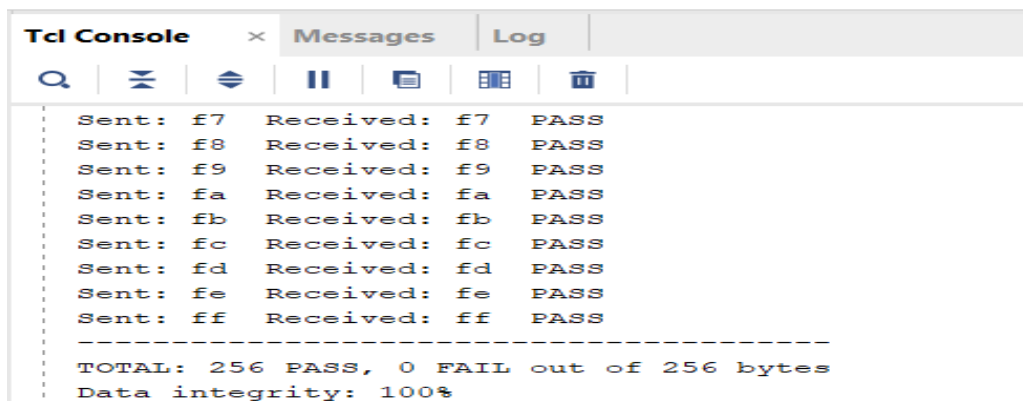


Figure 2: TCL Console output — 256-byte loopback test (100% data integrity)

7. Constraints File

A Xilinx Design Constraints (.xdc) file was created to define the primary clock for the design. The constraint specifies a 20 ns period (50 MHz) on the clk_50m port, which drives all four modules. No I/O placement constraints were required as the design was synthesised and implemented without a physical board.

```
# uart_constraints.xdc
create_clock -period 20.000 -name clk_50m [get_ports clk_50m]
```

8. FPGA Implementation Results

The design was synthesised and implemented in Xilinx Vivado targeting the Zynq-7000 device (xc7z010clg400-1). Default synthesis and implementation strategies were used. The following sub-sections report resource utilization, timing, and power results.

8.1 Resource Utilization

Resource	Used	Available	Utilization (%)
LUT	49	17,600	0.28%
Flip-Flop (FF)	56	35,200	0.16%
IO	33	100	33.00%
BUFG	1	32	3.13%

The design is extremely lightweight, occupying less than 1% of available LUT and FF resources. This is consistent with the minimal combinational logic required by the two FSMs and the baud rate counters.

Utilization			
		Post-Synthesis	Post-Implementation
Graph Table			
Resource	Utilization	Available	Utilization %
LUT	49	17600	0.28
FF	56	35200	0.16
IO	33	100	33.00
BUFG	1	32	3.13

Figure 3: Post-Implementation Resource utilization

8.2 Timing Analysis

Timing was analyzed with a 20 ns clock constraint. All timing paths met the constraint with a positive Worst Negative Slack (WNS).

Metric	Value
Clock Period (constraint)	20.000 ns (50 MHz)
Worst Negative Slack (WNS)	+15.471 ns
Total Negative Slack (TNS)	0 ns
Failing Endpoints	0
Maximum Frequency (Fmax)	$1 / (20 - 15.471) \text{ ns} = 220 \text{ MHz}$

The large positive slack indicates the design is not the critical path on this device, and could reliably operate at up to 220 MHz — well above the 50 MHz operating requirement.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 15.471 ns	Worst Hold Slack (WHS): 0.154 ns	Worst Pulse Width Slack (WPWS): 9.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 91	Total Number of Endpoints: 91	Total Number of Endpoints: 57

All user specified timing constraints are met.

Figure 4: Timing Summary — WNS = +15.471 ns, Fmax = 220 MHz

8.3 Power Analysis

Post-implementation power was estimated by Vivado's power analyser.

Component	Power
Total On-Chip Power	0.093 W
Junction Temperature	26.1°C
Thermal Margin	58.9°C (5.0 W)

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.093 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 26.1°C
 Thermal Margin: 58.9°C (5.0 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 11.5°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power

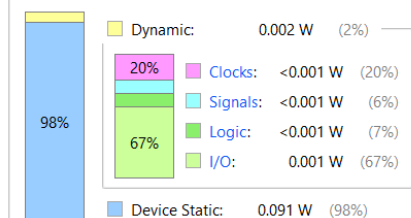


Figure 5: Power Analysis Summary

9. Discussion

The simulation and synthesis results confirm that the UART transceiver design is functionally correct, resource-efficient, and timing-clean on the Zynq-7000 platform.

The key design decision was the choice of 16x oversampling in the receiver. By sampling each incoming bit 16 times and using the middle sample (at Rxclk pulse 15) as the data value, the receiver tolerates up to $\pm 6.25\%$ baud rate mismatch between transmitter and receiver before a sampling error could occur. In the loopback configuration, transmitter and receiver share the same baud generator, so there is zero mismatch — but the oversampling also provides robustness against start-bit detection glitches.

The original receiver design from the project contained a subtle timing bug: after receiving a byte it returned directly to the START state rather than an explicit IDLE state, causing the sample counter to be non-zero at the beginning of the next start-bit search. This was corrected by introducing a dedicated IDLE state that cleanly resets all state before monitoring for the next falling edge. This fix was validated by the 256-byte loopback test achieving 100% data integrity.

The Fmax of 220 MHz demonstrates that the UART logic itself is not a performance bottleneck on this FPGA family. The design could easily support higher baud rates if the clock source were adjusted accordingly.

10. Conclusion

This project successfully implemented and verified a full-duplex UART transceiver in synthesisable Verilog. The design operates at 115,200 baud from a 50 MHz clock using 16x oversampling for robust data reception. Loopback simulation confirmed 100% data integrity across all 256 byte values. Post-implementation results on the Zynq-7000 FPGA show 49 LUTs, 56 FFs, 220 MHz maximum frequency, and 0.093 W total on-chip power — confirming suitability for resource-constrained embedded applications.

Key Achievements

- Full-duplex UART at 115,200 baud with 50 MHz clock, 16x oversampled receiver
- 4-state FSM transmitter and receiver, independently verified
- 256-byte loopback test: 256 PASS, 0 FAIL (100% data integrity)
- Post-implementation: 49 LUTs, 56 FFs, <1% resource utilization on Zynq-7000
- Timing: WNS = +15.471 ns, Fmax = 220 MHz (all constraints met)
- Power: 0.093 W total on-chip power

References

Xilinx Inc. Vivado Design Suite User Guide: Synthesis. UG901. Available at:
<https://www.xilinx.com/support/documentation>

Xilinx Inc. 7 Series FPGAs Data Sheet: Overview. DS180. Available at:
<https://www.xilinx.com>

IEEE Standard for Verilog Hardware Description Language. IEEE Std 1364-2005.

SP3 Spartan-3 FPGA Datasheet, UART Design Example. Xilinx Application Note XAPP223.